

Асинхронное программирование в Python

Асинхронное программирование позволяет выполнять несколько задач «одновременно», не дожидаясь завершения каждой по очереди. Это особенно полезно при работе с сетевыми запросами, базами данных и файлами — операциями, где программа обычно просто ждёт ответа. В Python для этого используется модуль `asyncio`.

Синхронное и асинхронное выполнение

В синхронном коде операции выполняются строго по порядку: пока одна не завершится, следующая не начнётся. Если операция «блокирующая» (например, ожидание ответа от сервера), вся программа простаивает.

В асинхронном коде задачи могут чередоваться: пока одна ждёт результата операции ввода-вывода (**IO, Input/Output** — чтение и запись данных: файлы, сеть, база данных), программа переключается на выполнение другой задачи, не блокируясь. Это называется **неблокирующим выполнением**.

Синхронное выполнение	Асинхронное выполнение
Код выполняется последовательно, шаг за шагом	Задачи чередуются между собой
Долгая операция блокирует всю программу	Ожидание одной задачи не блокирует другие
Проще писать, но медленнее при большом числе IO-операций	Быстрее там, где много ожидания (сеть, файлы, БД)

Где это применяется:

- Веб-серверы — обработка множества запросов одновременно, без блокировки на каждом.
- Приложения с интерфейсом — интерфейс остаётся отзывчивым во время долгих операций (загрузка данных, вычисления).
- Сетевые операции — HTTP-запросы, вебсокеты для чатов и другой связи в реальном времени.

Event loop (цикл событий)

В основе асинхронного подхода лежит **Event loop** — единый обработчик событий приложения. Устроено это так:

- приложение содержит один главный цикл событий (Event loop);
- «события» — это корутины (coroutine), аналог потоков для асинхронного кода;
- все события обрабатываются по единому стандартному протоколу;
- события хранятся в очереди, откуда их забирает обработчик;
- результатом события может быть новое событие, которое снова попадает в очередь.

Как только корутина доходит до операции ввода-вывода, она передаёт управление обратно Event loop, не блокируя программу, а возвращается к выполнению, когда результат готов.

Основы модуля asyncio

Начиная с Python 3.5 появились ключевые слова `async` и `await`.

- **Coroutine (корутина)** — аналог потока для асинхронного кода. Функция становится корутиной, если перед её определением стоит `async`.
- Вызов корутины напрямую не выполняет её код, а создаёт объект `coroutine object`. Выполнение всегда управляется Event loop.
- `await` приостанавливает выполнение корутины до готовности результата другой корутины. Состояние программы при этом сохраняется, и выполнение продолжается с той же точки.
- `asyncio.run()` — создаёт Event loop, запускает переданную корутину и завершает цикл, когда очередь опустеет. Это рекомендуемый способ запуска.

```
import asyncio

async def baz() -> str:
    print('Before Sleep')
    await asyncio.sleep(1)
    print('After Sleep')
    return 'Hello world'

async def main():
    r = baz()
    print(r)
    result = await r
    print(result)

if __name__ == '__main__':
    asyncio.run(main())
```

```
<coroutine object baz at 0x00000211E82CCDD0>
Before Sleep
After Sleep
Hello world
```

Вызов `print(r)` выводит объект-корутину, а не результат. Чтобы получить сам результат, нужен `await` — только после `result = await r` переменная получает значение `'Hello world'`.

Ускорение IO-задач с помощью `async/await`

Главное преимущество `async/await` — ускорение операций ввода-вывода. Синхронный вариант — три «запроса» по 0,5 секунды каждый выполняются по очереди:

```
from time import sleep, time

def get_user_sync(uid: int) -> dict:
    sleep(0.5)
    # ... поиск пользователя по id
    return user

for i in range(1, 4):
    print(get_user_sync(i))
# итого ≈ 1.5 секунды
```

Асинхронный вариант выполняет все три «запроса» параллельно с помощью `asyncio.gather()`:

```
import asyncio

async def get_user_async(uid: int) -> dict:
    await asyncio.sleep(0.5)
    # ... поиск пользователя по id
    return user

async def main():
    r = []
    for i in range(1, 4):
        r.append(get_user_async(i))
    return await asyncio.gather(*r)
# итого ≈ 0.5 секунды — втрое быстрее
```

`asyncio.gather()` ставит несколько корутин в очередь и позволяет Event loop выполнять их «параллельно» (по факту — чередуя между собой). Результаты возвращаются в том же порядке, в каком корутины были переданы.

Задачи (Task)

Обычно напрямую работать с низкоуровневыми примитивами `asyncio` не нужно. Но при необходимости корутину можно отправить в Event loop как отдельную задачу с помощью `asyncio.create_task()` — это вернёт объект `Task`.

`Task` — обёртка над корутиной, которая отслеживает момент её завершения. Задача начинает выполняться сама по себе (не дожидаясь `await`), а с помощью `await task` можно дождаться и получить её результат через `task.result()`.

```
import asyncio

async def random_value():
    print("start task")
    await asyncio.sleep(1)
    print("task finished")
    return random.random()

async def main():
    task = asyncio.create_task(random_value())
    print("task scheduled")
    await task
    print(f"result: {task.result()}")
```

```
task scheduled
start task
task finished
result: 0.4314553905867432
```

Задача создаётся сразу (строка `create_task`), но начинает фактически выполняться только когда Event loop получает управление — например, при `await task`.

Обработка исключений в задачах

Если внутри задачи возникает исключение, программа не падает сразу — исключение «сохраняется» в задаче и будет поднято только тогда, когда вы попытаетесь получить результат через `await task`.

```
async def faulty_task():
    raise ValueError("Ошибка в задаче")

async def main():
    task = asyncio.create_task(faulty_task())
    try:
        await task
    except ValueError as e:
        print(f"Исключение: {e}")
```

Отмена задач

Задачу можно прервать методом `task.cancel()`. При отмене внутри корутины возникает исключение `asyncio.CancelledError`.

```

async def long_running_task():
    try:
        await asyncio.sleep(10)
        return "Завершено"
    except asyncio.CancelledError:
        print("Задача была отменена")
        raise

async def main():
    task = asyncio.create_task(long_running_task())
    await asyncio.sleep(1)
    task.cancel()
    try:
        await task
    except asyncio.CancelledError:
        print("Задача отменена")

```

Важно: исключение `CancelledError` нужно правильно обрабатывать внутри корутины, чтобы корректно закрыть ресурсы (файлы, сетевые соединения). Необработанная отмена может привести к утечке ресурсов.

Callback-функции

Метод `add_done_callback()` назначает функцию, которая будет вызвана автоматически после завершения задачи. Полезно, когда нужно выполнить дополнительное действие (логирование, очистку ресурсов, уведомление), не дожидаясь задачи через `await`.

```

async def my_task():
    return "Результат"

def my_callback(task):
    print("Готово:", task.result())

async def main():
    task = asyncio.create_task(my_task())
    task.add_done_callback(my_callback)
    await task

```

Группировка задач: `gather()` и `wait()`

Функция	Когда использовать
<code>asyncio.gather()</code>	Нужно выполнить несколько задач параллельно и получить все результаты списком
<code>asyncio.wait()</code>	Нужен больший контроль: дождаться только части задач или отдельно обработать незавершённые

`gather()`

`asyncio.gather(*coros_or_futures, return_exceptions=False)` запускает несколько корутин параллельно, ждёт завершения всех и возвращает список результатов в порядке вызова. Если `return_exceptions=True`, исключения возвращаются как результаты вместо того, чтобы прерывать выполнение.

```

async def main():
    results = await asyncio.gather(task_one(), task_two())
    print(results)
    # ['результат task_one', 'результат task_two']

```

`wait()`

`asyncio.wait(futures, *, timeout=None, return_when=asyncio.ALL_COMPLETED)` возвращает два множества: завершённые (`done`) и незавершённые (`pending`) задачи.

- `timeout` — время в секундах, после которого функция вернёт управление, даже если не все задачи завершены;
- `return_when` — когда возвращать управление: `FIRST_COMPLETED` (первая завершённая), `FIRST_EXCEPTION` (первая с ошибкой) или `ALL_COMPLETED` (все, по умолчанию).

```
async def main():
    tasks = [asyncio.create_task(t) for t in [task_one(), task_two()]]
    done, pending = await asyncio.wait(tasks, return_when=asyncio.FIRST_COMPLETED)
    for task in done:
        print("Завершено:", task.result())
# Завершено: результат task_one
```

Таймауты: `asyncio.wait_for()`

Чтобы ограничить время выполнения задачи и избежать «зависания» программы, используется `asyncio.wait_for()`. Она принимает корутину (или `Task`) и время в секундах. Если задача не успевает завершиться, возникает исключение `asyncio.TimeoutError`.

```
async def long_running_task():
    await asyncio.sleep(5)
    return "Завершено"

async def main():
    try:
        result = await asyncio.wait_for(long_running_task(), timeout=3)
        print(result)
    except asyncio.TimeoutError:
        print("Задача не завершена вовремя")
```

`wait_for()` полезен там, где важно не дать программе бесконечно ждать сетевой запрос или другую долгую операцию, сохраняя отзывчивость системы.

Параллельное выполнение CPU-bound и IO-bound задач

Не все пакеты с операциями ввода-вывода (**IO, Input/Output**) поддерживают синтаксис `async/await`. Если внутри асинхронного кода нужно вызвать именно такую — блокирующую — функцию, используют пакет `concurrent.futures` и метод `run_in_executor` объекта `loop` (это и есть Event loop).

Пример: синхронная функция `blocks` выполняет тяжёлые вычисления (**CPU-bound** — задача, упирающаяся в производительность процессора, а не в ожидание), а параллельно с ней работает асинхронный мониторинг:

```

import asyncio
import concurrent.futures

def blocks(n):
    # тяжёлые вычисления — блокирующая функция
    ...

async def monitoring():
    while True:
        await asyncio.sleep(2)
        print(f'Monitoring {time()}')

async def run_blocking_tasks(executor, n):
    loop = asyncio.get_event_loop()
    result = await loop.run_in_executor(executor, blocks, n)
    return result

async def main():
    asyncio.create_task(monitoring())
    with concurrent.futures.ThreadPoolExecutor(max_workers=3) as executor:
        futures = [run_blocking_tasks(executor, n) for n in [50_000_000, 60_000_000, 70_000_000]]
        return await asyncio.gather(*futures)

```

Функцию `blocks` оборачивают в `run_blocking_tasks`: внутри берут текущий `loop` и через `run_in_executor` получают асинхронный результат — сам блокирующий вызов при этом выполняется в отдельном потоке пула `ThreadPoolExecutor`, не блокируя Event loop. Пока три тяжёлые задачи считаются в потоках, корутина `monitoring` продолжает работать параллельно.

Для действительно тяжёлых CPU-вычислений вместо `ThreadPoolExecutor` можно использовать `ProcessPoolExecutor` — тогда вызовы выполняются в отдельных процессах и не мешают друг другу (в отличие от потоков, которые в Python ограничены GIL).

Тот же приём подходит для «утяжеления» обычных синхронных сетевых запросов (например, через `requests`, у которого нет своего `async`-API):

```

async def preview_fetch_async():
    loop = asyncio.get_running_loop()
    with ThreadPoolExecutor(3) as pool:
        futures = [loop.run_in_executor(pool, preview_fetch, url) for url in urls]
        return await asyncio.gather(*futures, return_exceptions=True)

```

Функцию `preview_fetch` (обычный синхронный запрос через `requests`) не переписывают — её просто помещают в `run_in_executor` для каждого URL, а список полученных `Future` передают в `asyncio.gather()`. Даже на трёх запросах разница в скорости заметна.

Параметр `return_exceptions` у `gather()`: по умолчанию `False` — первое же исключение сразу прерывает ожидание. При `True` исключения не прерывают выполнение, а попадают в список результатов наравне с успешными.

Асинхронная работа с файлами

Для файловых операций тоже существуют асинхронные пакеты — стандартная функция `open` в Python блокирующая и в асинхронном коде замедляет Event loop.

aiofile

Вместо `open` используется `async_open`, возвращающая файлоподобный объект с асинхронными методами: `read()`, `write()`, `readline()`, а также `seek()` / `tell()` для позиции указателя.

```
import asyncio
from aiofile import async_open

async def main():
    async with async_open("hello.txt", 'w+') as afp:
        await afp.write("Hello world\n")

    # чтение целиком:
    async with async_open("hello.txt", 'r') as afp:
        print(await afp.read())

    # построчно:
    async with async_open("hello.txt", 'r') as afp:
        async for line in afp:
            print(line)
```

Для построчного чтения больших файлов метод `readline` у `async_open` неэффективен (не переиспользует буфер). Лучше использовать помощник `LineReader` — он читает файл фрагментами (по умолчанию 4 КБ) и сам находит границы строк:

```
from aiofile import AIOFile, LineReader

async with AIOFile("hello.txt", 'r') as afp:
    async for line in LineReader(afp):
        print(line)
```

aiopath

Асинхронный аналог стандартного `pathlib` — тот же API, но все методы асинхронные. Удобен, когда не хочется смешивать блокирующие и неблокирующие операции с путями и файлами.

```
from aiopath import AsyncPath

async def main():
    apath = AsyncPath("hello.txt")
    print(await apath.exists())
    print(await apath.is_file())
```

aioshutil

Асинхронная версия стандартного модуля `shutil` (копирование, перемещение файлов и т.п.). Обычный `shutil` в асинхронном коде блокирует Event loop — `aioshutil` даёт неблокирующие аналоги тех же функций.

```
from aiopath import AsyncPath
from aioshutil import copyfile

async def main():
    apath = AsyncPath("hello.txt")
    if await apath.exists():
        new_path = AsyncPath('logs')
        await new_path.mkdir(exist_ok=True, parents=True)
        await copyfile(apath, new_path / apath)
```

Асинхронные HTTP-запросы: aiohttp

`aiohttp` — асинхронный фреймворк для работы с HTTP поверх `asyncio`. В нём есть и клиентская, и серверная часть; внешние зависимости не нужны. Устанавливается командой `pip install aiohttp`.

Выполнение запроса

Особенность `aiohttp` — обязательное использование сессий соединения (`ClientSession`). У многих других библиотек сессия скрыта для простоты, но это заметно снижает производительность — каждый запрос создаёт новое соединение. В `aiohttp` запрос всегда выполняется в контексте сессии: методы `session.get()`, `.post()`, `.put()` и другие.

Именованные аргументы запроса: `data` (тело запроса в байтах), `params` (словарь query-параметров), `json` (объект, который будет отправлен как JSON), `headers` (заголовки), `cookies`.

```
import aiohttp
import asyncio

async def main():
    async with aiohttp.ClientSession() as session:
        async with session.get('https://python.org') as response:
            print("Status:", response.status)
            html = await response.text()
            print("Body:", html[:15], "...")

asyncio.run(main())
```

На Windows перед `asyncio.run()` иногда нужно выставить `asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())`, чтобы избежать ошибки `RuntimeError: Event loop is closed`.

Обработка ответа сервера

Заголовок ответа гарантированно доступен сразу же — а вот тело может быть большим (файл, поток данных), поэтому `aiohttp` позволяет сначала посмотреть только заголовок и решить, стоит ли грузить тело целиком.

Атрибуты ответа (`response`): `status` — код ответа сервера; `headers` — словарь заголовков; `cookies` — куки, если были переданы; `ok` — `True`, если статус меньше 400.

Методы получения тела: `read()` — `bytes`; `text()` — строка (для `text/plain`); `json()` — автоматически распарсенный JSON-объект.

```
async with session.get('https://api.privatbank.ua/p24api/pubinfo?json&exchange&coursid=5') as response:
    print(response.status, response.ok)
    result = await response.json()
    # [{'ccy': 'USD', 'base_ccy': 'UAH', 'buy': '38.9', 'sale': '39.4'}, ...]
```

Работа с сессиями

Рекомендуемый способ — менеджер контекста (`async with`): он гарантированно закрывает соединения. Но сессию можно открывать и закрывать вручную:

```
session = aiohttp.ClientSession()
response = await session.get('https://python.org')
html = await response.text()
response.close()
await session.close()
```

Хорошая практика — использовать одну сессию для всех запросов к одному сервису: это заметно ускоряет их выполнение. Сессию можно передать как аргумент в несколько корутин и выполнить их параллельно через `gather()`:

```

async def index(session):
    async with session.get('https://python.org') as response:
        return await response.text()

async def doc(session):
    async with session.get('https://www.python.org/doc/') as response:
        return await response.text()

async def main():
    async with aiohttp.ClientSession() as session:
        return await asyncio.gather(index(session), doc(session))

```

Оба запроса выполняются «одновременно» в рамках одной сессии — суммарное время близко к времени самого долгого из запросов, а не к сумме обоих.

Сессию можно настроить один раз — заданные параметры (заголовки, таймаут) автоматически применяются ко всем запросам этой сессии:

```

timeout = aiohttp.ClientTimeout(total=1)
async with aiohttp.ClientSession(
    headers={"Request-Id": str(uuid4())},
    timeout=timeout,
) as session:
    ...

```

Другие HTTP-методы

Помимо `get()`, у сессии есть `post()`, `put()`, `delete()`, `head()`, `options()`, `patch()` — используются аналогично.

```

async with session.post('http://localhost:5000', data={"message": "Hello world!"}) as response:
    print(response.status)          # 201
    print(await response.text())    # Done request!message=Hello+world%21

```

Обработка ошибок

Два типа ошибок при запросах нужно обрабатывать по-разному:

- **Ошибочный статус ответа** (например, 404) — сервер ответил, но неудачно. Проверяется обычным условием.
- **Ошибка соединения** (не тот хост, нет сети) — сервер вообще не ответил. Такие ошибки поднимают исключение `aiohttp.ClientConnectorError` и требуют `try/except`.

```

try:
    async with session.get(url) as resp:
        if resp.status == 200:
            html = await resp.text()
        else:
            print(f"Error status: {resp.status} for {url}")
except aiohttp.ClientConnectorError as err:
    print(f'Connection error: {url}', str(err))

```